

CTU Practice

Code exercises, Weeks 1–3

Contents

Week 1 — Fundamentals

Week 2 — Conditionals

Week 3 — Lists, Tuples, and Loops

Week 1 — Fundamentals

Concepts: print/input, type conversion, expressions, float formatting, debugging

Easy

1. Write a script with fixed variables `daily_rate = 35`, `days = 6`, `weeks = 2` that computes and prints the total pay for a pet-sitter, using one `print()` call combining a label and the result.
2. Write a script that asks the user for their name and favorite color with two separate `input()` calls, then prints both on the same output line, separated by a comma (use `end=' '` at least once).
3. Convert the hard-coded string `"250"` into an integer, add `50` to it, and print both the original string and the new integer, each with its own label.

Medium

4. Ask the user for a temperature in Celsius, convert it to a `float`, and print the equivalent Fahrenheit temperature ($F = C * 9/5 + 32$) formatted to exactly 2 decimal places.
5. Write a script with an intentional syntax error inside a comment (mismatched quotes in a `print()` call), then the corrected, working version below it.
6. Ask for three separate purchase amounts (as input, converted to `float`) and print their average formatted to 2 decimal places, labeled `"Average purchase: $X.XX"`.
7. Write a script with a logic bug: it's meant to apply a 10% discount but multiplies the price by `10` instead of `0.10`. Leave the buggy version commented above the fixed, working version.

Hard

8. Write a program that reads `weight_kg` and `height_m` (both floats, from input), computes BMI ($\text{weight_kg} / \text{height_m} ** 2$), and prints the result formatted to 2 decimal places with a labeled message.
9. Write a program that reads three separate inputs for hours worked on three different days, sums them using an expression wrapped across multiple lines inside parentheses (implicit continuation), multiplies by an hourly wage of `18`, and prints the weekly pay.
10. Write a program that reads `distance_km` (float, input) and uses `speed_kmh = 27000` to compute how many days a trip would take; print the result once formatted to 4 decimal places, and again formatted to 2 decimal places, each with its own label.

Week 2 — Conditionals

Concepts: if/elif/else, boolean logic, nested ifs, indentation, dictionary lookups

Easy

1. Write a script that reads `movie_rating` (int, input) and prints "Kid-friendly" if it's 3 or higher, otherwise "Not for kids".
2. Write a script that reads `club_member_years` (int, input) and sets `discount = 15` if 5 or more years, otherwise `discount = 0`; print the result.
3. Write a script with a multi-branch `if/elif/else` that reads `grade_percent` (int, input) and prints "A" for 90+, "B" for 80-89, "C" for 70-79, and "F" otherwise.

Medium

4. Write a script using `or` that prints "Lucky number!" if a user-entered number is 7, 13, or 21, otherwise "Just a number.".
5. Write a script using `and` that prints "Approved" only if `age` (input, int) is 18 or older AND the user answered 'yes' to having an ID, otherwise "Denied".
6. Write a script with several **independent** `if` statements (not `elif`) that reads `product_price` (input, float) and prints each message that applies: "Eligible for free shipping" if price > 50, "Eligible for member discount" if price > 100, "Premium item" if price > 200 — more than one message can print for the same input.
7. Write a script with a nested `if` inside an `if/else`: read whether it's raining and whether the user has an umbrella (both compared against 'yes'); if raining, print a different message depending on the umbrella; if not raining, print one message regardless.

Hard

8. Write a program checking a small hard-coded inventory dictionary (item name -> stock count) against a user-entered item name, with a nested `if/else` inside an outer membership check — first write a version with **intentionally broken/inconsistent indentation** as a comment, then the corrected, working version below it.
9. Write a program that reads three separate input integers and, using only `if/elif/else` (no `min()`, `max()`, or `sorted()`), determines and prints the **median** (middle value) of the three.
10. Write a program that reads a city name (input) and looks it up in a hard-coded dictionary of city populations; if found, use a nested `if` to print "Large city" (population > 1,000,000) or "Small city" otherwise; if not found in the dictionary, print "Unknown city".

Week 3 — Lists, Tuples, and Loops

Concepts: lists, tuples, **while** and **for** loops, sentinel values, nested loops, looping over strings and dictionaries

Easy

1. Create a list of five snacks, print the whole list, then print the first, third, and fifth items using their index.
2. Create `steps = [4200, 8100, 6500]`, change the second element to `9000`, append `7300`, then print the list and how many items it holds using `len()`.
3. Create a tuple of the five weekdays (`'Mon'`, `'Tue'`, `'Wed'`, `'Thu'`, `'Fri'`) and use a **for** loop to print `"Today is <day>"` for each one.

Medium

4. Given `ratings = [4.5, 3.0, 5.0, 2.5, 4.0]`, use a **for** loop with an accumulator variable to compute the sum, then print the average formatted to 2 decimal places, plus the lowest and highest rating using `min()` and `max()`.
5. Read a savings `goal` (float, input). Starting from `balance = 500.0`, use a **while** loop that adds 8% interest each year until the balance reaches the goal; print the year number and balance (2 decimals) on every iteration, then the total years it took.
6. Read words from the user one per line until they type `done` (a sentinel value that should not be stored). Store the words in a list with `.append()`, then print how many words were entered and print the words in reverse order using `reversed()`.
7. Given `stock = {'apples': 3, 'pears': 0, 'plums': 7}`, use a **for** loop over the dictionary to print `"<fruit>: <n> in stock"` for each entry; inside the same loop, count how many fruits are out of stock (value `0`) and print that count after the loop ends.

Hard

8. Read `n` (int, input) and use nested **for** loops with `range()` to print an `n` by `n` multiplication table, one row per line, values separated by spaces (use `end=' '` and an empty `print()` at the end of each row).
9. Read a sentence from the user and use a **for** loop over its characters to count vowels, consonants (letters that are not vowels), and digits separately, ignoring everything else (spaces, punctuation); print the three counts.
10. Read integers into a list until the user enters `0` (the sentinel, not stored). Then, without using `sum()`, use a **for** loop to compute the total and count the even numbers; finally print the average (2 decimals) and, in a second **for** loop, every number in the list that is greater than that average. Handle the case where the user enters `0` immediately (empty list) without crashing.